

Ch#2:

Lexical Analyzer

- ⇒ Lexical analyzer is the first phase of compiler construction.
- ⇒ it is also known as scanner.
- ⇒ it scans high level language that is pure characters by characters and converts it into a sequence of Tokens.
- ⇒ It takes Lexemes as input and produces Tokens.

Functions

- ⇒ it reads pure HLL characters by characters that's why it is called scanner.
- ⇒ it groups the lexemes into valid tokens. ^{eliminating non tokens} that is called Tokenization. ^{removes comments}
- ⇒ it removes white space characters.
- ⇒ it removes comments! (//, #)
- ⇒ it helps to identify token into the symbol table. Token ->
- ⇒ it also generates lexical errors. and gives the line no of the error.
- if it encounters invalid Token.
- ⇒ The lexical analyzer works in two Phases:

⇒ it performs scan in the first phase.
 ⇒ It does lexical analysis in the 2nd phase.

:Siti

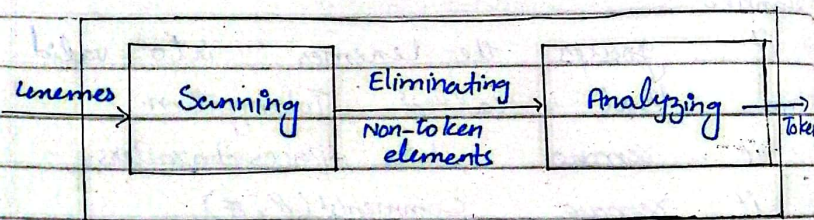
Types of Tokens

basically describe the class or category of input string

⇒ There are seven different types of Tokens

- Literals
- Identifiers
- Operators
- Constants
- Keywords
- Literals
- Punctuations

:Siti

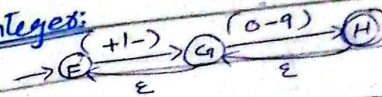


C-Tokens:

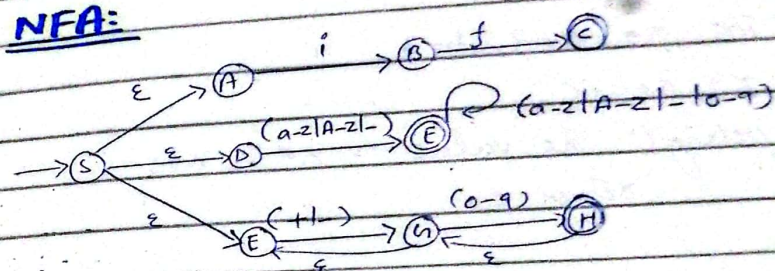
if: (A) --i--> (B) --f--> (C)

identifier: (D) --(a-zA-Z)--> (E) --(a-zA-Z|_|0-9)--> (E)

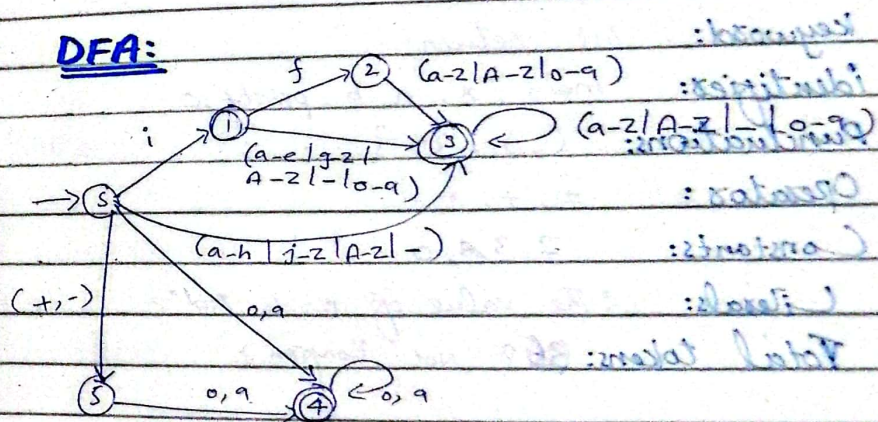
Integers:



NFA:



DFA:



:promis

Tokenization:

To Count the number of Tokens:

```
int main(-)
{
    int x, a=2, b=3;
    x = a+b*c;
    printf("The value is %d", x);
    return 0;
}
```

keyword: int, return

identifiers: main, x, a, b, printf, c

Punctuations: (,), {, }, ., ;

Operators: =, +, *

Constants: 2, 3, 0

Literals: "The value of x is %d"

Total tokens: 36? - not confirm

Summary:

⇒ Scanning is the first phase of a three-part process that the compiler uses to understand the i/p program.

⇒ The scanner reads the input stream of characters and produces a stream of words.

⇒ It aggregates characters to form words and applies a set of rules to determine whether or not each word is legal or not in the source language.

⇒ If the word is valid, the scanner assigns it a syntactic category or part of speech.

⇒ The scanner compiles to touch every character in the i/p program.

What is recognizer?

⇒ It is a program that identifies specific words in a stream of characters.

⇒ It takes an input string x and answers "yes" if x is a sentence of the language and "no" otherwise.

⇒ The regular expression is compiled into a recognizer by constructing a generalized transition diagram called a finite automata.

e.g. FA is the recognizer of R.E.

What is syntactic category?

- ⇒ A classification of words according to their grammatical usage.
- ⇒ In a practical sense, these categories correspond to terminal symbols in the language grammar.
- ⇒ Scanner aggregates characters into words.
- ⇒ For each word, it determines if the word is valid in the source program.

Q. What is microsyntax?

- ⇒ The lexical structure of a language is called microsyntax.

Q. What is Flex?

Q. Generate the appropriate tokens for given piece of code.

```
int MAX (int a, int b)
{
```

```
    if (a > b)
```

```
        return a;
```

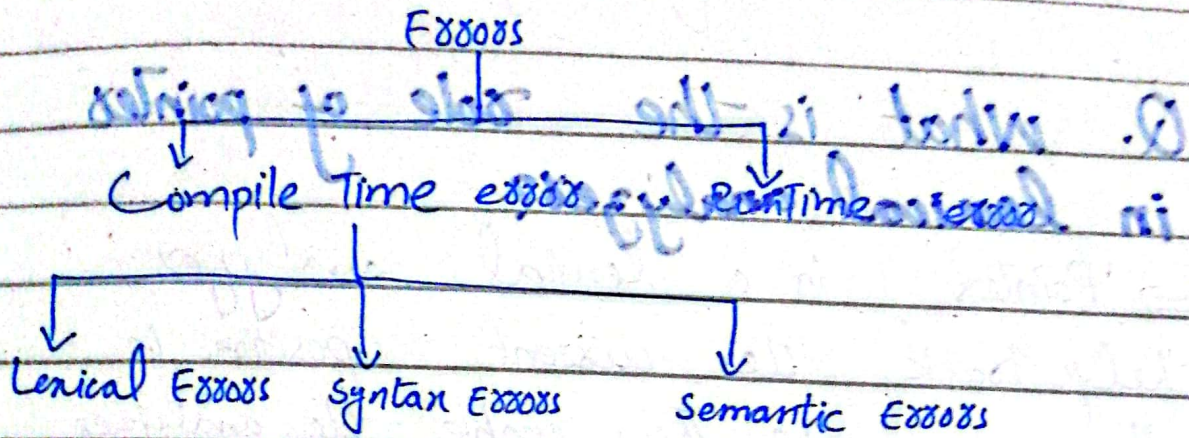
```
    else
```

```
        return b;
```

```
}
```

Lexemes	Tokens	Lexeme	Tokens
int	keyword	(	Operator
MAX	identifier	a	identifier
(	Operator	>	Operator
int	keyword	b	identifier
a	identifier	)	Operator
,	Operator	return	keyword
int	keyword	a	identifier
b	identifier	;	Punctuation
)	Operator	else	keyword
{	Operator	return	keyword
if	keyword	b	identifier
		;	Punctuation
		}	Operator

Classification of Errors



Identifiers allowed to significance length:-
($\therefore$ 311 912

C: 31/247

C++: 2048

Python: 79

Lexical errors: $\Rightarrow$ A character sequence which is not possible to scan any valid token is a lexical error.

⇒ Identifiers ^{which is a lexical error} that are way too long

Spontaneous breaking of discrete symmetry

⇒ Numeric Constants which are ill-formed

=> Illegal characters that are absent from the source code. Spelling errors.

Lexical Excess-Redundancy:

- o Panic mode recovery
- o Transpose of two adjacent characters.
- o Insert a missing character.
- o Delete unknown or extra character
- o Replace a character with another

⇒ Lexical phase processes the execution of program.

Q. What is the role of pointers in lexical analyzer?

⇒ Pointers in a lexical analyzer help track the current position in the source code. They enable the analyzer to read characters and group them into tokens like keywords and identifiers.

⇒ By updating the pointers, the analyzer accurately identifies and extracts tokens, making it a crucial part of the compilation process.

Q. How can we represent a token?

⇒ A token is typically represented using a data structure that contains two main pieces of information.

⇒ The token type.

⇒ And its value.

Type: The type specifies the category of the token (such as keyword, identifiers, literal, operators) etc.

values in the value table. The content of the table is the source code.

Q. Regular expressions or FA are used in which phase of compiler?

⇒ They are used in the lexical analysis phase of compiler.

Q. Perform the lexical analysis of the following statement.

```
if (idx == 0)
{
    idx = 50;
    *p++ = 1;
}
```

Ans:

if	(	idx	==	0	)	{	idx	=	50	;	}
keyword	OP	Var	OP	inter	OP	Var	OP	int	OP		

Q. Consider the following program

```
main()
{
    int z = 10;
    if (z < 50)
    {
        p = 55;
    }
}
```


How many lexical errors are present in program?

Zero errors

However, ~~some errors~~ ~~are~~ ~~not~~ ~~detected~~ ~~by~~ ~~the~~ ~~lexer~~ ~~but~~ ~~by~~ ~~the~~ ~~semantic~~ ~~analyzer~~

as "P" is not declared but ~~is~~ ~~used~~ ~~in~~ ~~the~~ ~~program~~.
Used. But no lexical errors.

Q. Consider the following program

```
#include <stdio.h>
#include <math.h>
int main()
{
    printf("%d + %d = %d", 3, 1, 4);
    /* sum */
    return 0;
}
```

How many lexemes in the program after preprocessing are?

In preprocessing, ~~macro~~ ~~preprocessor~~ ~~commands~~ ~~are~~ ~~removed~~.

~~int~~ ~~main~~ ~~{~~ ~~printf~~ ~~("~~ ~~%d~~ ~~+~~ ~~%d~~ ~~=~~ ~~%d~~ ~~",~~ ~~3~~ ~~,~~ ~~1~~ ~~,~~ ~~4~~ ~~);~~ ~~/*~~ ~~sum~~ ~~*/~~ ~~return~~ ~~0;~~ ~~}~~

So, there are 20 lexemes after preprocessing.

Find the number of tokens in the following C code using lexical analyzer of compiler.

```
main()
{
    int *a, b;
    B=10;
    A=&b;
    printf("%d %d", b, *a);
    B = /* pointer */ b;
}
```

In above lines, ~~/* pointer */~~ ~~will~~ ~~not~~ ~~be~~ ~~counted~~ ~~as~~ ~~token~~.

token	count	token	count	token	count
main	1	:	8	:	15
}	2	&	9	=	16
int	3	:	10	/	17
a	4	+	11	:	18
b	5	.	12	)	19
B	6	,	13	{	20
10	7	a	14	*	21

,	22	point	29
;	23	"%d %d"	30
=	24	b	31
*	25	*	32
)	26	)	33
b	27	b	34
*	28	b	35
		}	36

Hence, Total of 36 tokens exist.

Q. Count the number of tokens in the following code

```
switch(inputvalue)
```

```
{
```

```
case 1: {
```

```
b = c * d;
```

```
break;
```

```
default:
```

```
b = b ++;
```

```
break;
```

```
}
```

Token	value	c	"
switch	1	*	12
(	2	d	13
inputvalue	3	;	14
)	4	break	15
{	5	:	16
case	6	default	17
1	7	:	18
:	8	b	19
b	9	=	20
=	10	c	21
		++	22
		:	23
		break	24
		:	25
		}	26

Hence, there are 26 total numbers of tokens.

Q. Count the number of tokens in given code?

```
printf("i=%d, &i=%x", i, &i);
```


Point	1	subor
(	2	
"i=1-1, & i=1-1"	3	
?	4	
i	5	
,	6	
&	7	
i	8	
)	9	
:	10	

Here, there are 10 tokens.

modat go sedmuni all- trua
 (is, i, "is, b-i") thing

Short term

initials - 3

Complete FA:

⇒ An FA explicitly includes all error transitions is called Complete FA.

AFA

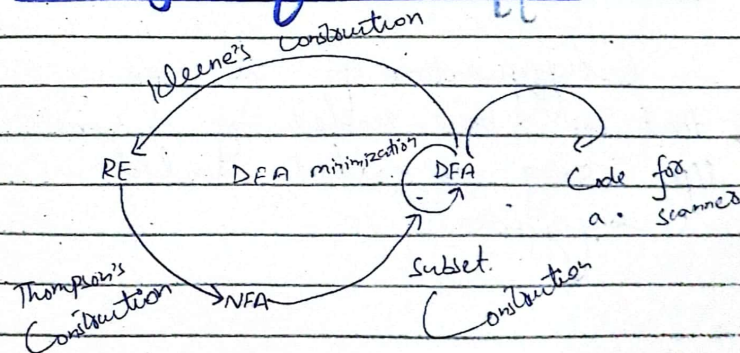
Regular expressions:

⇒ Regular expressions are a concise and powerful notation for specifying the microsyntax of programming languages.

⇒ R.E are builds on three basic operations over finite alphabets:

- Alternation
- Concatenation
- Kleene Closure

The Cycle of Construction



miset book

E-transition:

⇒ A transition on the empty string, ϵ that does not advance the input.

DFA:

⇒ A DFA is an NFA whose transition function is single-valued. DFA do not allow ϵ -transition.

Configuration of an NFA:

⇒ The set of concurrently active states of an NFA is called the configuration of an NFA.

Valid Configuration:

⇒ Configuration of an NFA that can be reached by some i/p string is called valid configuration.

Monotone function:

⇒ A function f on domain D is monotone iff, $\forall x, y \in D, x \leq y \Rightarrow f(x) \leq f(y)$

Set partition:

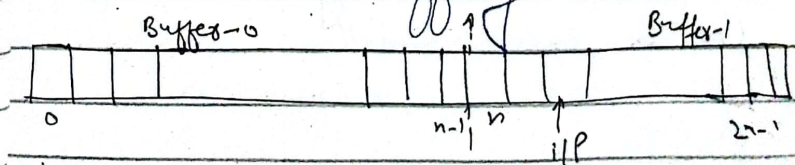
⇒ A set partition of S is a collection of non-empty, disjoint subsets of S whose union is exactly S .

Collating Sequence:

⇒ The "sorting order" of the characters in an alphabet, determined by the integers assigned each character is called collating sequence.

Double buffering:-

⇒ A scheme that uses two i/p buffers in a modulo fashion to provide bounded rollback is often called double buffering.



Q. What is Lex? Use of Lex?

⇒ Lex is a tool in lexical analysis phase to recognize tokens using regular expression.

⇒ Lex tool itself is written in Lex Compiler.

Use of Lex:

lex.l → Lex Compiler → lex.yy.c

lex.yy.c → Compiler → a.out

i/p stream → a.out → Sequence of tokens

⇒ It is commonly used with YACC (Yet Another Compiler Compiler).

It was written by Mike Lesk & Eric Schmidt.

Lex file format:

⇒ A Lex Program consists of three parts and is separated by % delimiters:-

• Declarations

%

• Translation rules

%

• Auxiliary procedures

Q. Write the regular expression for floating point numbers?

⇒ ~~[+|-]? ([0-9]*[.])? [0-9]+~~

~~1-([0-9]+ \. [0-9]+)~~

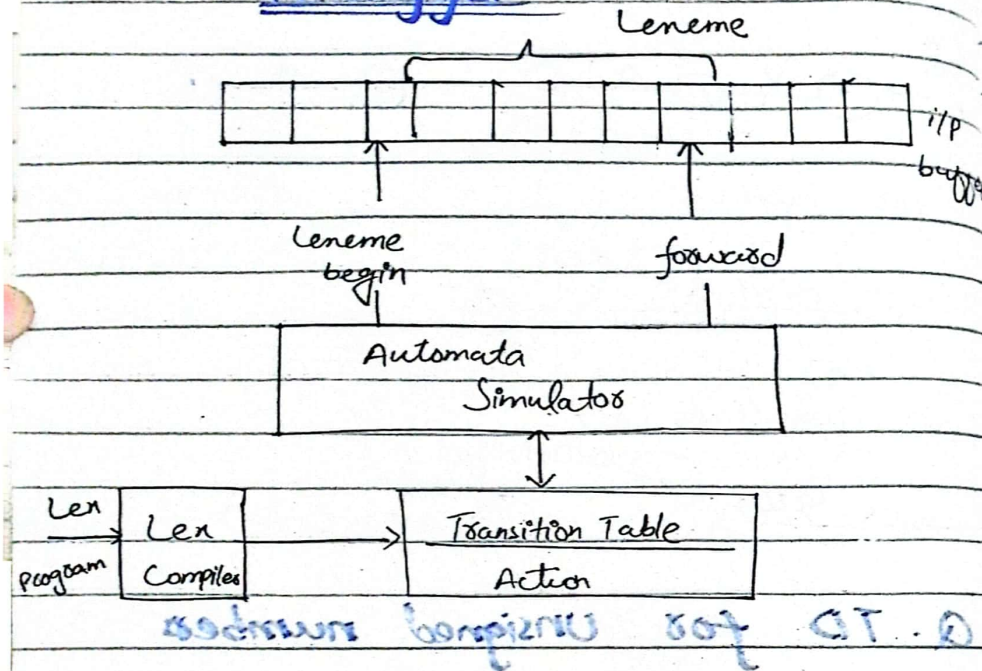
⇒ ~~[+|-]? ([0-9]+ ([.] [0-9]*)?)? [0-9]+~~

~~[+|-]? ([0-9]* \. ? [0-9]+ ([0-9]+)~~

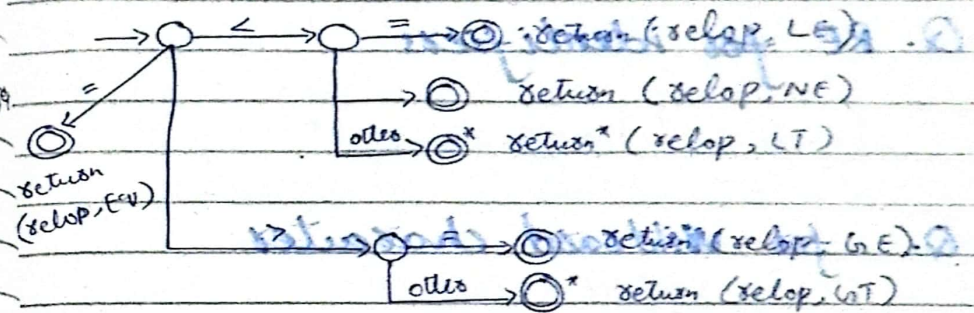
Q. Write the regular expression for identifiers that are strings of letters, underscores or digits beginning with a non-digit?

~~[A-Z | a-z | '] + [A-Z | a-z | 0-9 | -]*~~

Structure of Generated AEC Analyzer



Q. TG for Relational Operators
($<$, $<=$, $>$, $>=$, $=$, $<>$)



Q. RE for white spaces in AEC

$[\text{ } \backslash t \backslash n]^+$

Q. Regular expression for Unsigned numbers

digit $\rightarrow [0-9]$
digit $\rightarrow \text{digit}^+$

number = $\text{digits} [\cdot \text{digits}]^? [E [+ | -]^? \text{digits}]^?$

means or one zero

Q. RE for parentheses

Q. R-E for integer constants

$0 | ([1-9] [0-9]^*)$

Q. RE for simple operators
 (<, >, =, <=, >=, <=, >=)
 "*", "/", "+", "-"

Q. RE for identifiers

$[a-zA-Z_][a-zA-Z0-9_]*$

Q. for wildcard characters
 "." (dot);

Q. DFA for recognizing a single line comment



Q. How to go from RE to DFA

- ⇒ flex accepts a list of regular expressions (regen)
- ⇒ Converts each regen internally to an NFA (Thompson Construction);
- ⇒ Connect each NFA to DFA (subset construction)
- ⇒ may minimize DFA

Summary:

- ⇒ A scanner transforms a string of characters into a string of tokens
- ⇒ Scanners generating tools like flex allow you to define a regular expression for each type of token
- ⇒ Internally, the regular expressions are transformed to a deterministic finite automata for matching.
- ⇒ To break ties, matching uses principles:
 - o Longest match
 - o first match

Q. What is the O/P of lexical analyzer?

Tokens

Q. Why we use regular expressions to define the lexical syntax of a language?
⇒ There are several reasons:

- o Separating the syntactic structure of a language into lexical and non-lexical parts provides a convenient way of modularizing the front end of a compiler into two manageable-sized components.

- o The lexical rules of a language are frequently quite simple, and to describe them we do not need a notation as powerful as grammars.

- o Regular expressions generally provide a more concise and easier-to-understand notation for tokens than grammars.

- o More efficient lexical analyzers can be constructed automatically from regular expressions than from arbitrary grammars.